TECHNION
Israel Institute of Technology

DEPARTMENT OF CHEMICAL ENGINEERING
TECHNION – ISRAEL INSTITUTE OF TECHNOLOGY

WEEK 02 · LECTURE · 00560413

AI Fluency and Agentic Methods for Engineers

Lecture 2

How an LLM works, and how it fails

Assoc. Prof. Alon Grinberg Dana

Department of Chemical Engineering · Technion

Outline

01	**Field update**	what moved this week ←
02	LLM	what it is, how it is trained, how it fails
03	More git	worktrees, merge conflicts, GitHub

Field update

One model edited another model's weights

GPT-6 Astra read the numbers inside a small open model (Qwen3.5, 0.8 B), found **one direction** that made it loop, and subtracted it from the feed-forward output of two layers.

~~training~~ ~~data~~ ~~gradient descent~~

7.9% → 48.1%

MATH-500 score, after changing **0.016%** of the weights

By the end of today, you will know each word:

weights **vector space** **direction**

feed-forward **layer**

> Real, and over-hyped: the model mostly **stopped looping** until it ran out of tokens. It did not learn new maths. Critics say an engineer could have found the same edit by hand.

Takehi, Teoh, Zhang, Khattab (MIT CSAIL), “Sparks of Weight-Space Programming”, blog post, 2026-11-02, not peer reviewed · VentureBeat; R&D World

Outline

01	Field update	what moved this week
02	**LLM**	what it is, how it is trained, how it fails ←
03	More git	worktrees, merge conflicts, GitHub

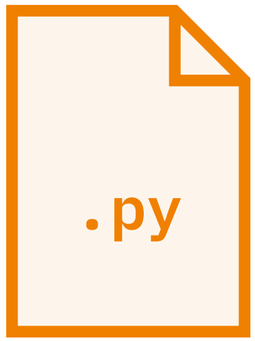
An LLM is two files

gpt-oss-120b · OpenAI · open weights · Aug 2025



`parameters`

117 billion numbers · ≈ 65 GB



`run`

a short program that uses them

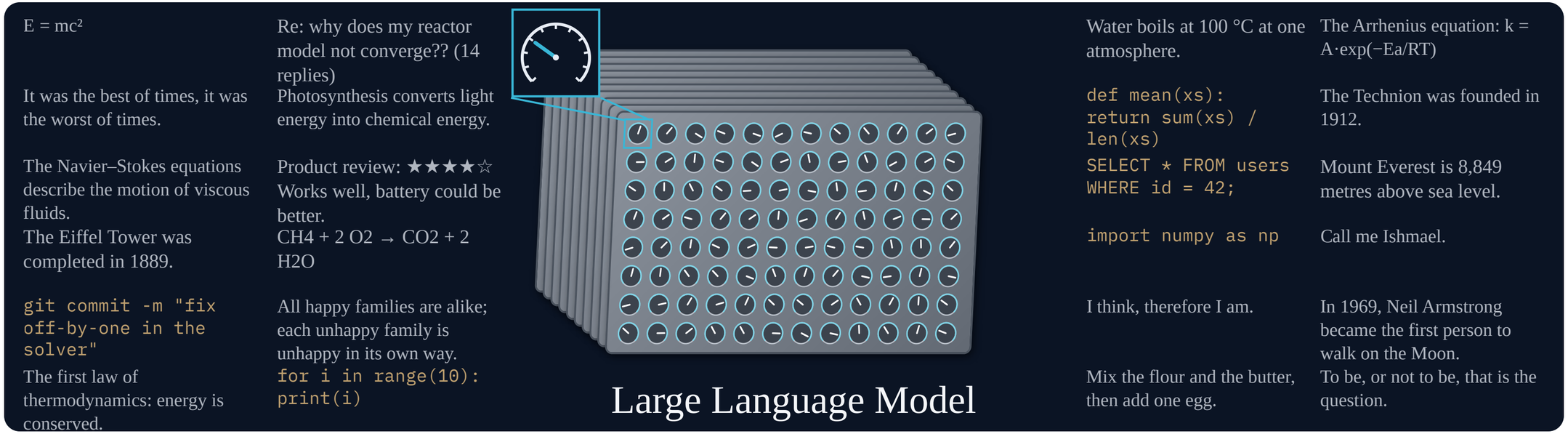
Runs on one 80 GB GPU, or a high-memory laptop. No internet needed.

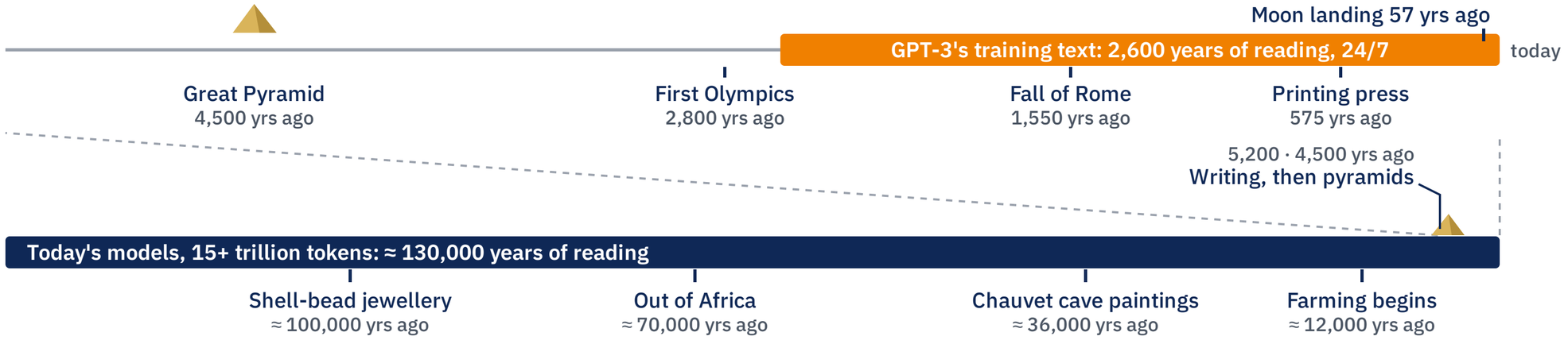
The strongest open model today, Kimi K3.1 Pro (Moonshot, Nov 2026):
2.8 trillion parameters, ≈ 1.56 TB. It needs a server, not a laptop.

gpt-oss model card, arXiv:2508.10925 · Kimi K3.1 Pro model card, Hugging Face (2026-11-04)

Training: the model reads the internet

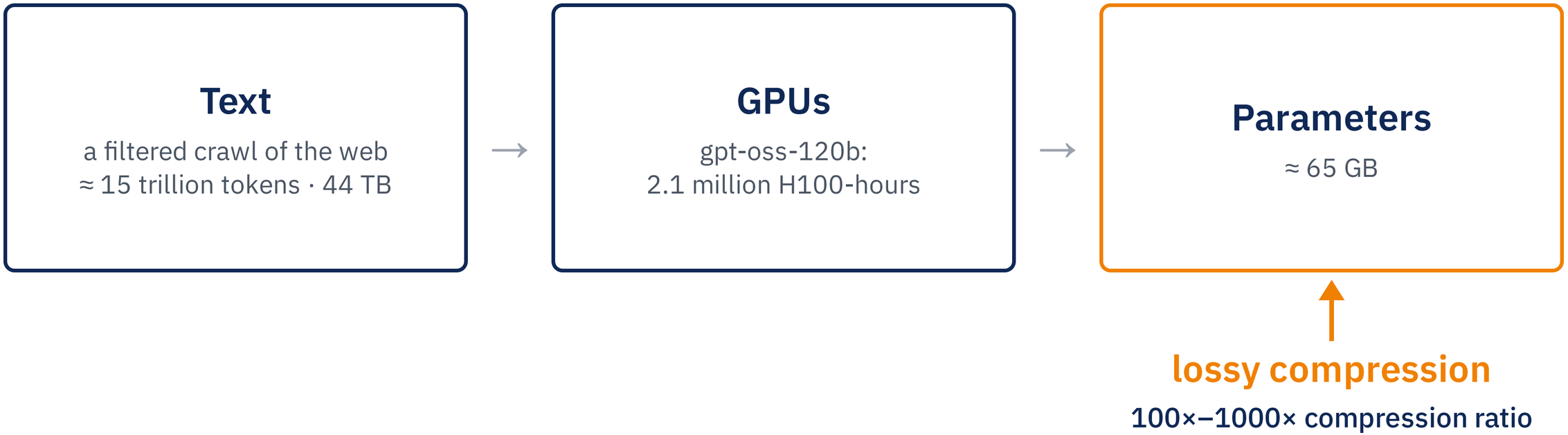
Trillions of tokens, of every kind





2,600 yrs: 3Blue1Brown (2024), GPT-3 ≈ 300 B tokens · 130,000 yrs: the same reading rate for 15 T tokens (Llama 3, 2024) · dates: round textbook values

Training: compress the internet



2023 · LLAMA-2-70B

10 TB of text · 6,000 GPUs · 12 days · ≈ $2 M

2026 · FRONTIER MODELS

Clusters of 100,000+ GPUs.
Cost not published, far beyond $2 M.

FineWeb (Hugging Face, 2024) · gpt-oss model card (2025) · Karpathy, Intro to LLMs (2023)

What the model actually reads

Tokens: chunks of text, each a number

Text	`Large` ` language` ` models` ` are` ` los` `sy` ` zip` ` files` ` of` ` the` ` internet` `.`	12
What the model gets	42565 6439 7015 553 1403 5611 16216 6291 328 290 6179 13	
A measurement	`Ea` ` =` ` ` `52` `.` `3` ` k` `J` `/mol` ` at` ` ` `298` `.` `15` ` K`	15
The same, in Hebrew	`.` `ינטרנט` `הא` `של` `יפ` `ז` `צי` `ב` `ק` `הם` `ולים` `גד` `פה` `ש` `לי` `מוד`	16

About 200,000 different tokens. A number is several tokens: 298 . 15 is 298 · . · 15.
Tokens are the unit of everything: the **price**, the **context window**, the **speed**.

o200k_base tokenizer (GPT-4o, gpt-oss), computed 2026-11-03

The one thing it does

Predict the next token, pick one, repeat

`the cat sat on a mat`

A probability for every token.
It **samples** one, appends it,
and runs again.



Sampling is a weighted coin: **same question, different answer.**

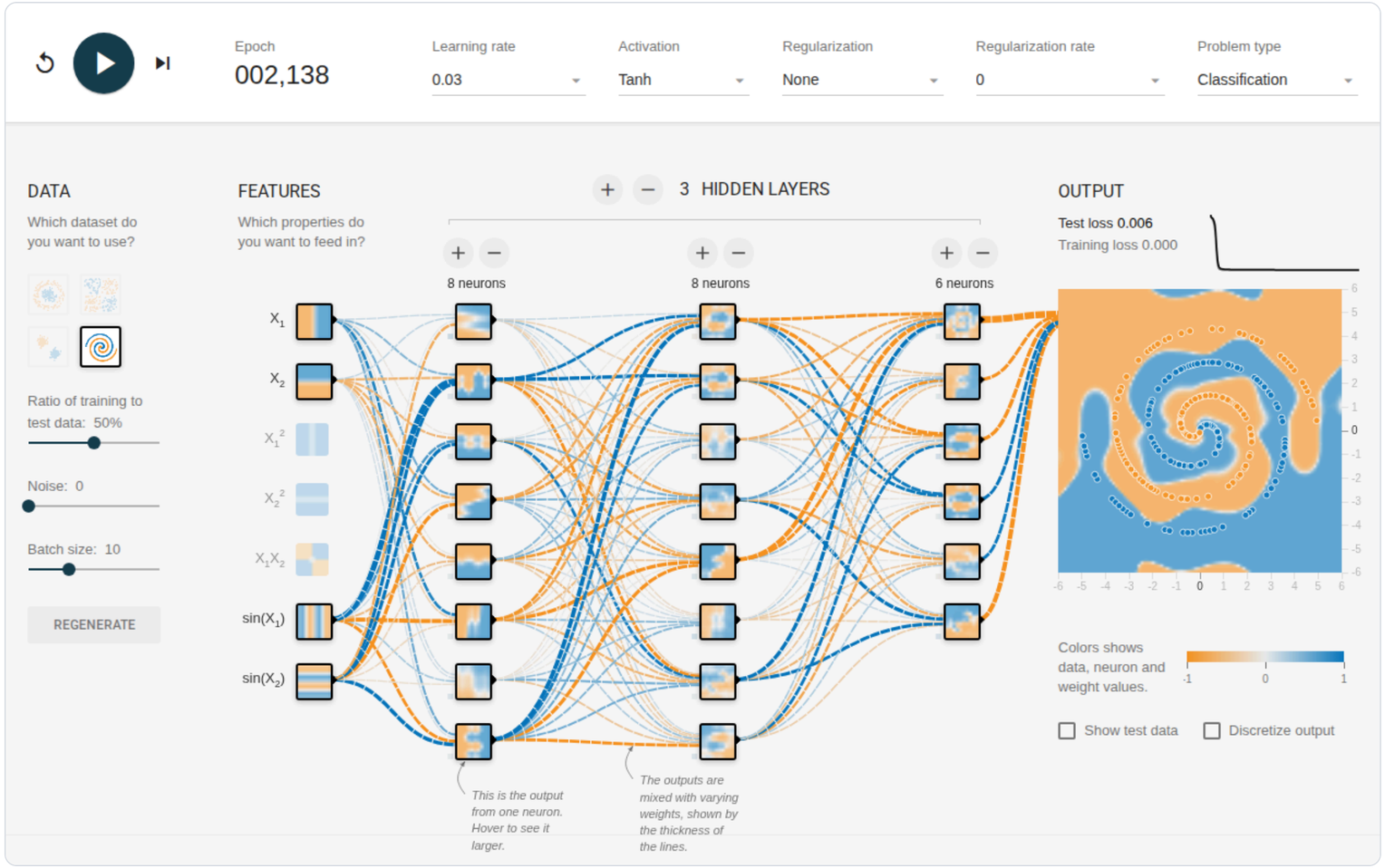
How training turns the dials

Back-propagation: one step



A neural network learns, live

Try it yourself: playground.tensorflow.org



Each line is a dial; its thickness is the size of the number.

Why prediction is powerful

The learning process forces the model to learn a lot about the world

Ilan Ramon

48 languages

Article Talk Read Edit View history

From Wikipedia, the free encyclopedia

Ilan Ramon (Hebrew: אילן רמון, pronounced [(ʔ)iˈlan ʁaˈmon]; June 20, 1954 – February 1, 2003)[1] was an Israeli fighter pilot and later the first Israeli astronaut. He served as a Space Shuttle payload specialist on STS-107, the fatal mission of *Columbia* in which he and the six other crew members were killed when the spacecraft disintegrated during re-entry. At 48, Ramon was the oldest member of the crew. He is the first foreign recipient of the United States Congressional Space Medal of Honor, which was awarded posthumously.[2]

Ilan Ramon
אילן רמון



Biography [edit]

Born as Ilan Wolfferman in Ramat Gan, Israel, to Tonya (1929–2003) and Eliezer Wolfferman (1923–2006), a family of Ashkenazi Jewish descent, Ramon grew up in Beersheba. His father was from Germany, and his family fled Nazi persecution in 1935. His mother and grandmother were from Poland, and were Holocaust survivors, having been in Auschwitz. They emigrated to Israel in 1949.[3] Ramon's first name, Ilan, means "tree" in Hebrew.[4] Ramon Hebraizied his surname from **Wolfferman** to **Ramon**, a more "Israeli (Sabra)" name, when he joined the Israeli Air Force (IAF). This was

To guess 1954 or Columbia, the network must hold facts about Ilan Ramon. Code, chemistry and grammar get in the same way.

Nobody programs the knowledge. It is a side effect of the objective.

Run it freely, and it dreams

Text shaped like the internet, true or not

Smith, J. A., & Levi, R. (2019). Kinetic modelling of methane oxidation in micro-combustors. *Combust. Flame*, 208, 112–124. doi:10.1016/j.combustflame.2019.06.017

Every part has the right form. The red parts are plausible numbers, not looked up.

1 SO IT FAILS BY fluent wrongness

It was trained for plausible continuation, not truth. Fluency is not evidence.

How does it work inside?

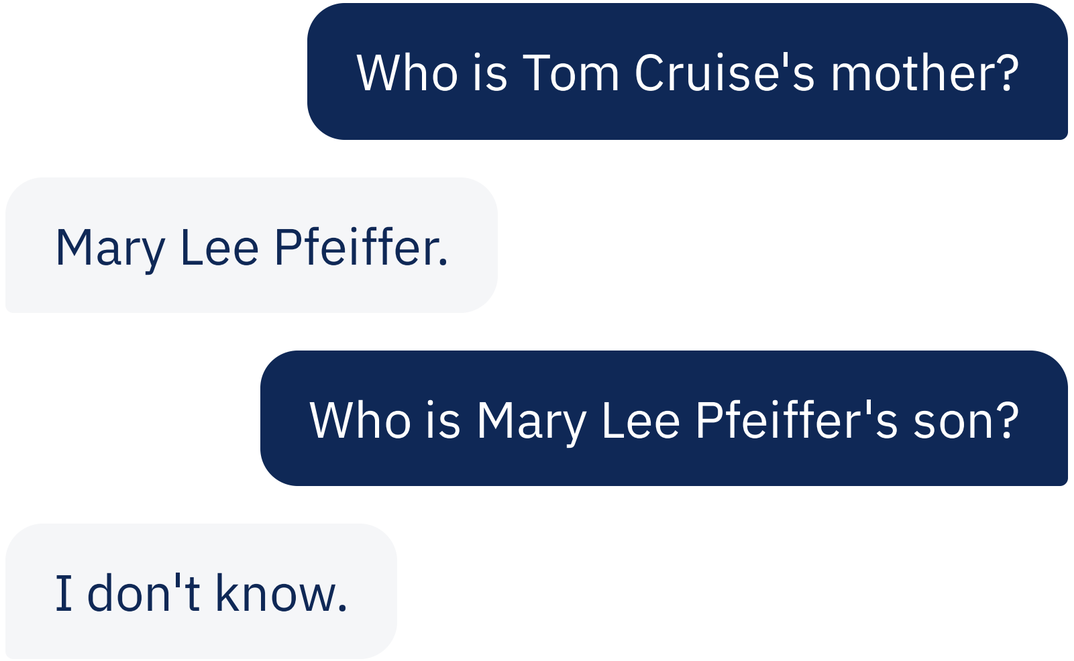
A transformer: we know the recipe, not the result

We know every operation in it.
We know how to nudge the numbers to predict better.

We do not know how billions of numbers combine into what it does.

Treat it as an empirical artifact: measure it.

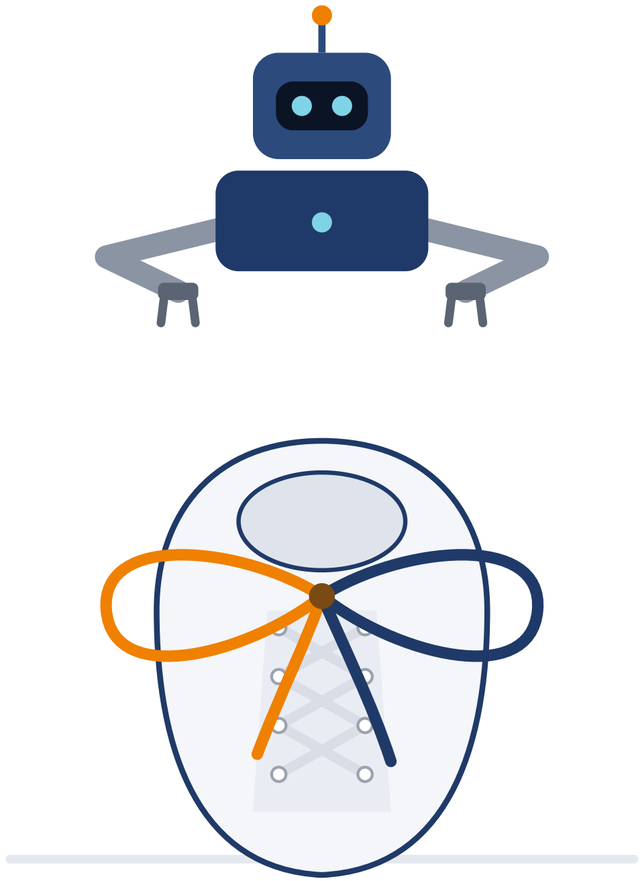
Example: the reversal curse



Berglund et al., The Reversal Curse (2023) · GPT-4, 2023

Nobody wrote the steps

A transformer learned to tie shoelaces from examples



Two robot arms, one transformer:
217 million learned numbers.

Trained on 5,133 human demonstrations.
No rule says “loop, then cross, then pull”.

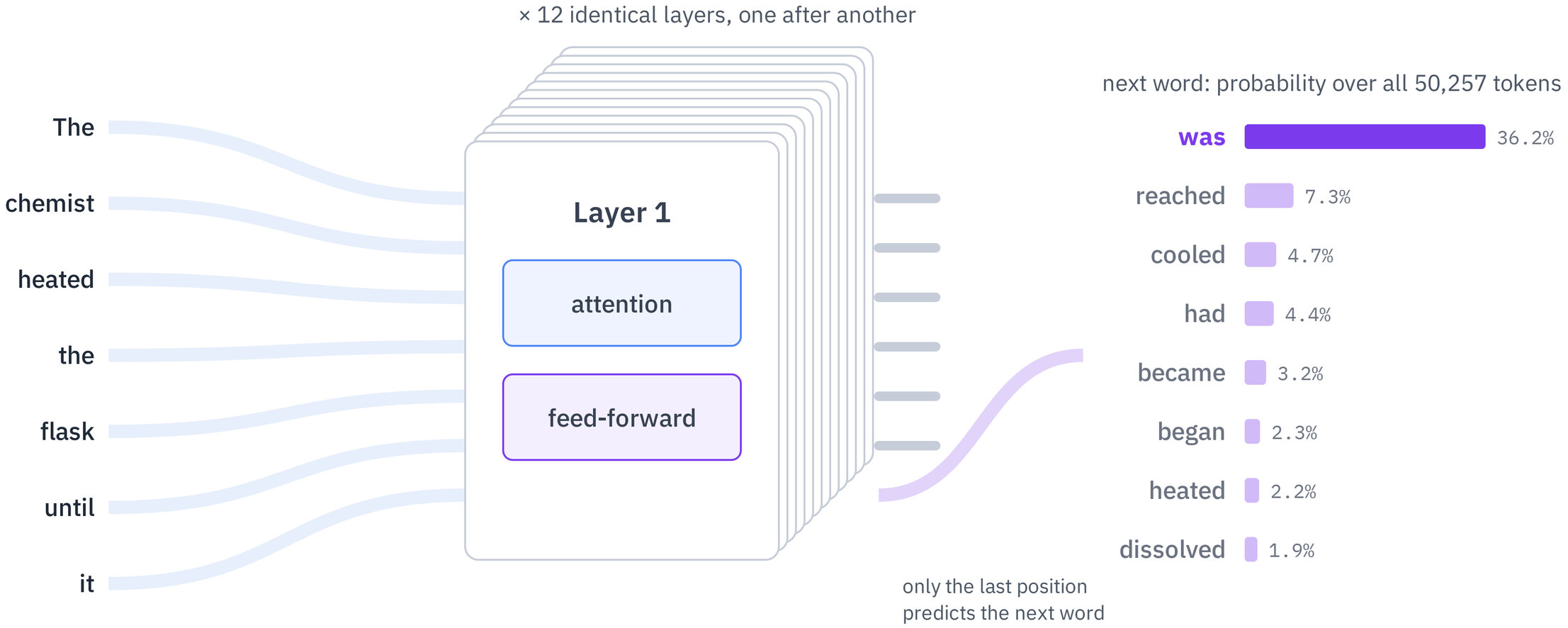
Engineers built the hands.
Training found the knot.

An LLM is the same deal. Next: open the hands.

Zhao et al., ALOHA Unleashed: A Simple Recipe for Robot Dexterity (Google DeepMind, CoRL 2024) · animation: schematic

A transformer, end to end

GPT-2 small: 124 million learned numbers, 12 identical layers



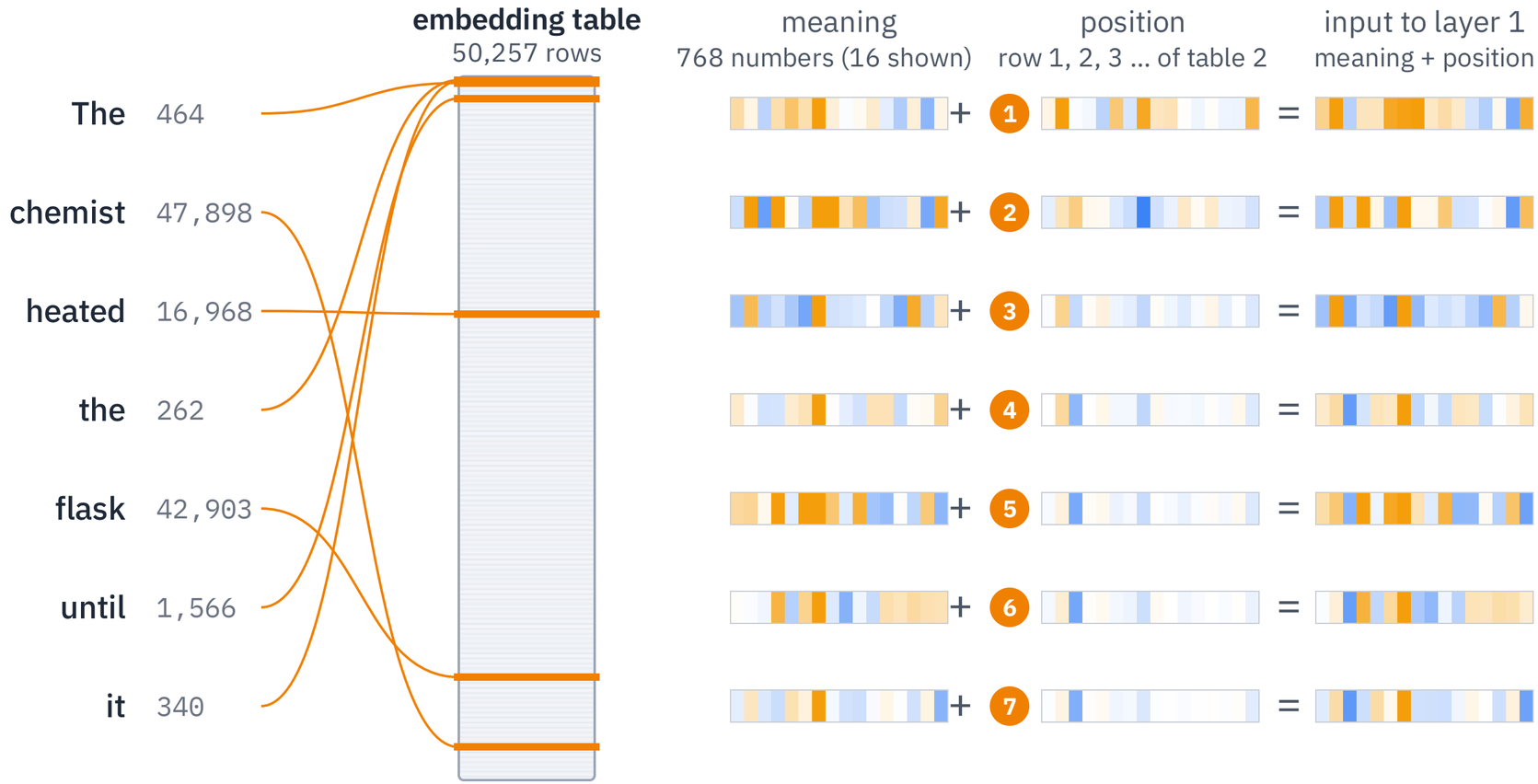
Text in, one next word out. The same 12 layers do all the work.

Vaswani et al., Attention Is All You Need (NeurIPS 2017): the transformer · GPT-2: Radford et al. (OpenAI, 2019)

Tokens become vectors

Real GPT-2 tokens, ids and embedding values

“The chemist heated the flask until it …”



Blue: positive.
Orange: negative.

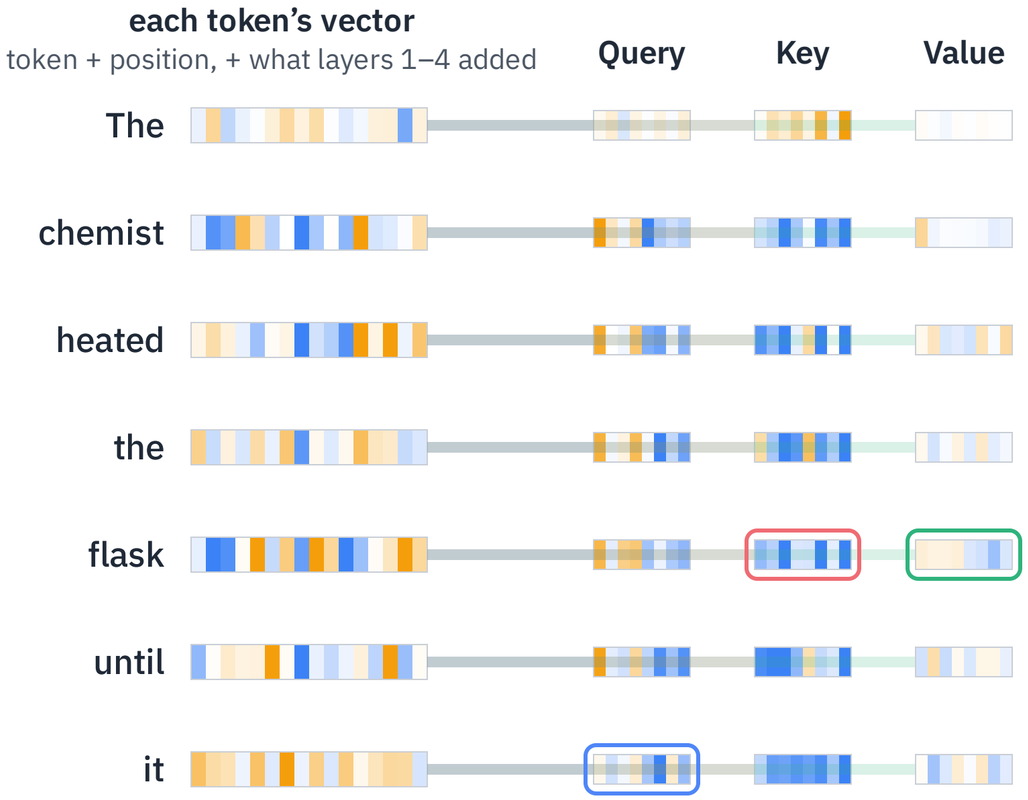
No single number means anything. Similar words/tokens get similar vectors.

“pyrolysis” → `py` `ro` `ly` `sis` 4 tokens

Add the position vector (row 1, 2, 3 …), number by number: word order now counts.

Query, Key, Value

One head, real numbers: layer 5, head 4 of 12



Query what am I looking for?
"it" asks: which singular thing came before me?

Key what I offer, as a label
"flask" shows: a singular object, a container

Value what I hand over if chosen
"flask" hands over its meaning

Like a library: search by subject (query ↔ key), then take the book (value), not the card.

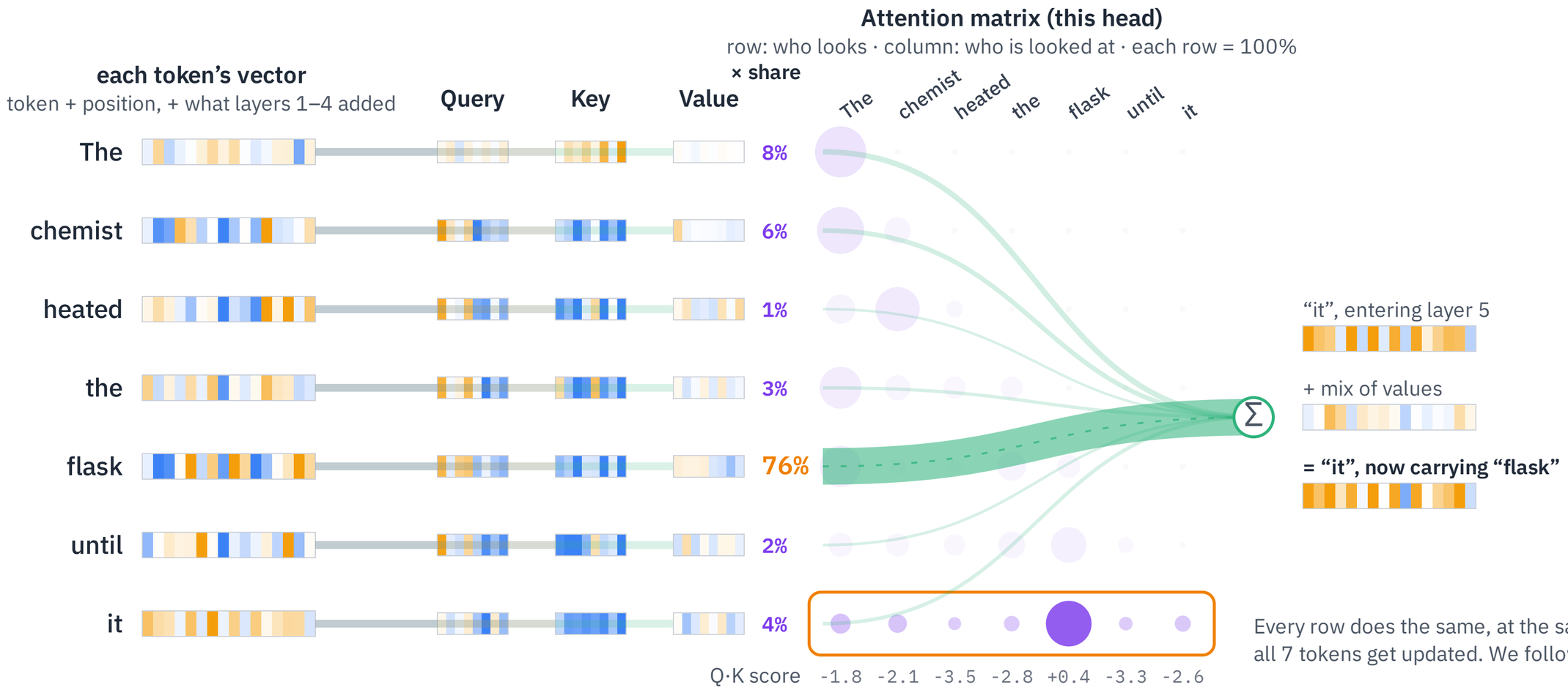
Learned, not labelled.

The three matrices are trained like every other dial. "Question", "label", "content" is our reading of them; in most heads, the numbers have no clear meaning.

Three learned matrices give each token a Query, a Key and a Value.

Attention: tokens talk to each other

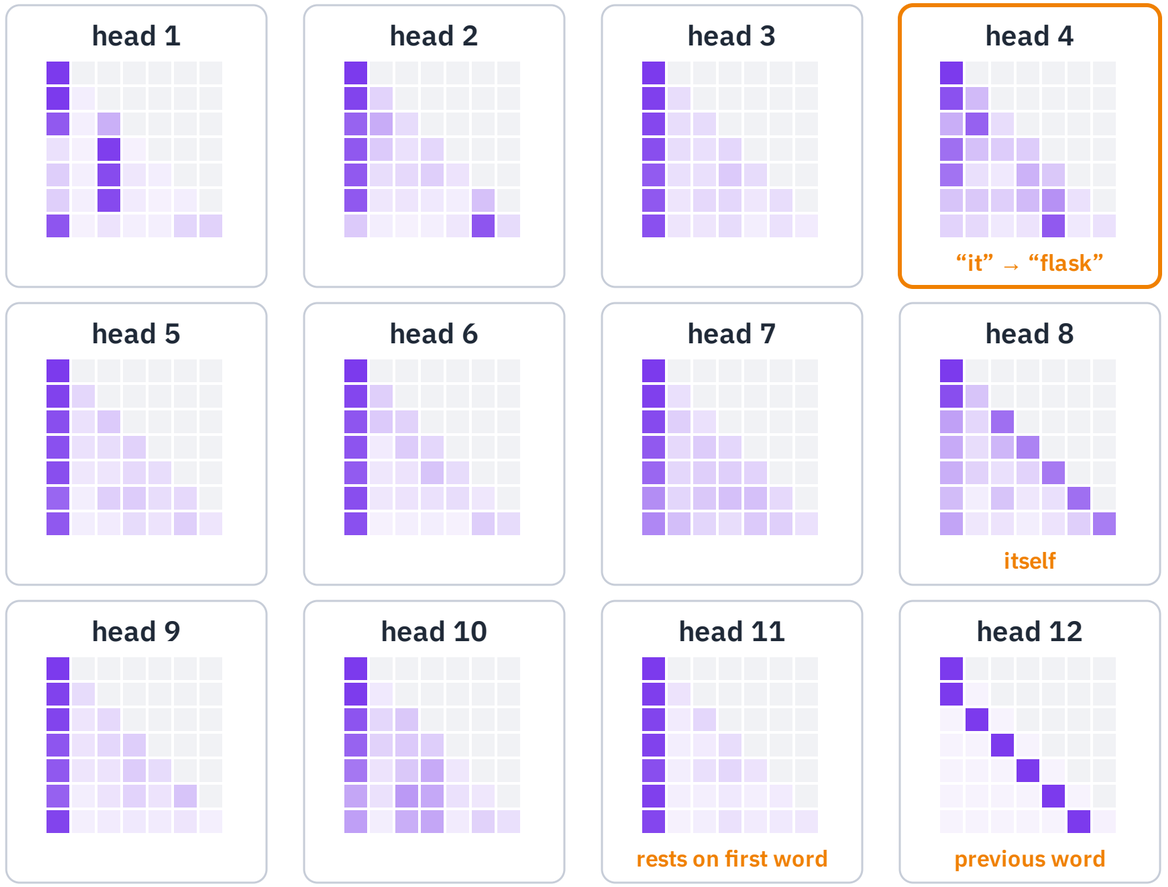
One head, real numbers: layer 5, head 4 of 12

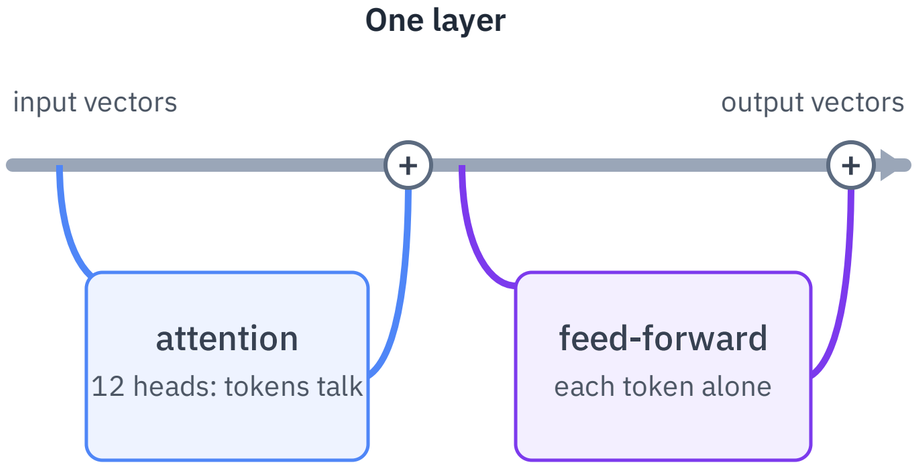


“it” takes the values, mixed by those shares, and adds them to itself.

Many heads, then think alone

All 12 heads of layer 5, on our sentence (real)





The thick grey line is the residual:
output = input + what the block computed.

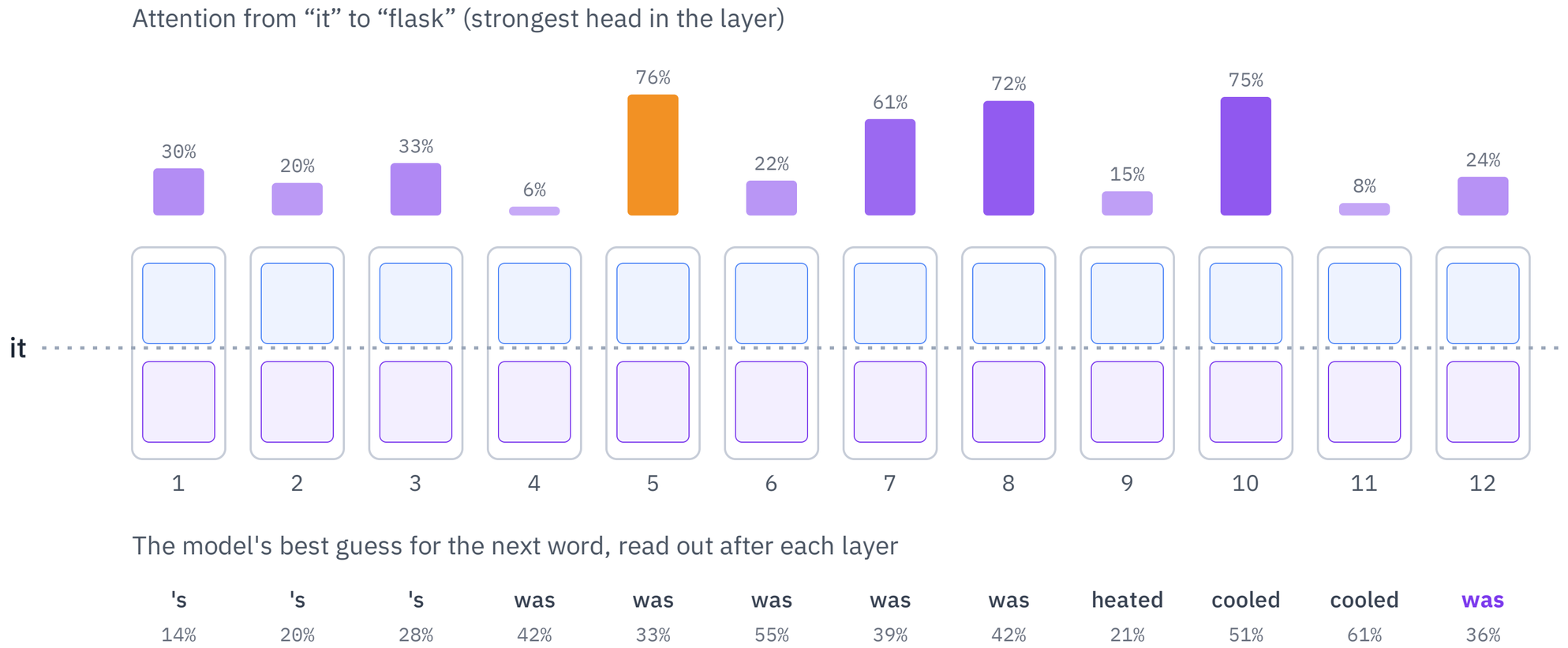
A token never loses what it was; it only gains.

Feed-forward holds ⅔ of each layer’s numbers, and most stored facts.

Then each token thinks alone. The residual line keeps what it already was.

Why so many layers?

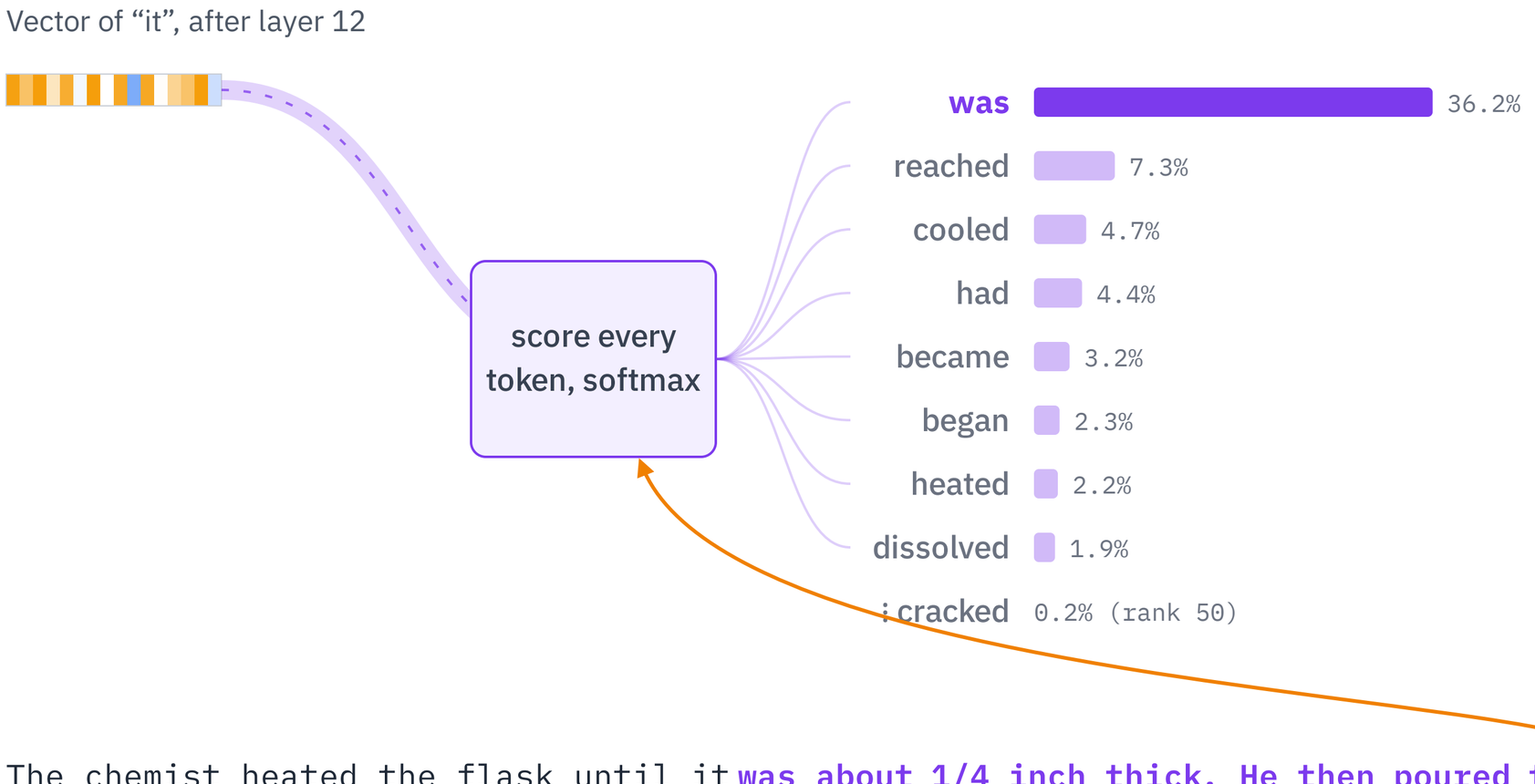
Each layer adds one more step on top of the last



Real readout: the guess forms, wavers (“cooled”), and settles on “was”.

Pick a word, append, repeat

Real GPT-2 small output for our sentence



`The chemist heated the flask until it` **`was about 1/4 inch thick. He then poured the`**

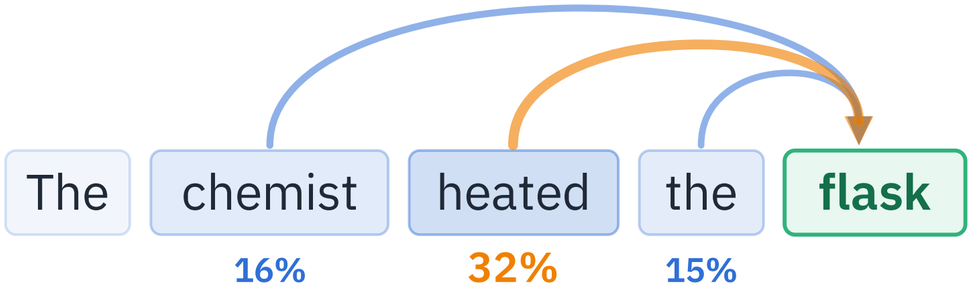
… and so on, until it predicts the stop token <|endoftext|> (token 50,256).

append, run the whole stack again

Step 5: pick one, append it, run the whole stack again. Real GPT-2 output.

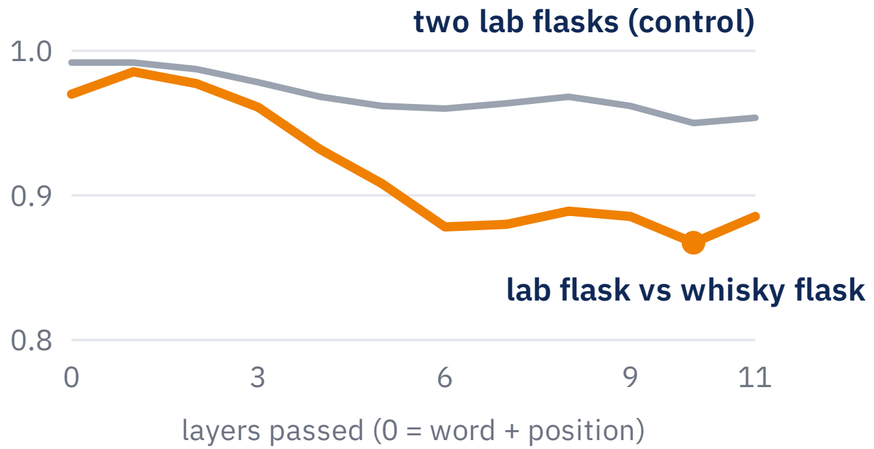
Attention in action: which flask?

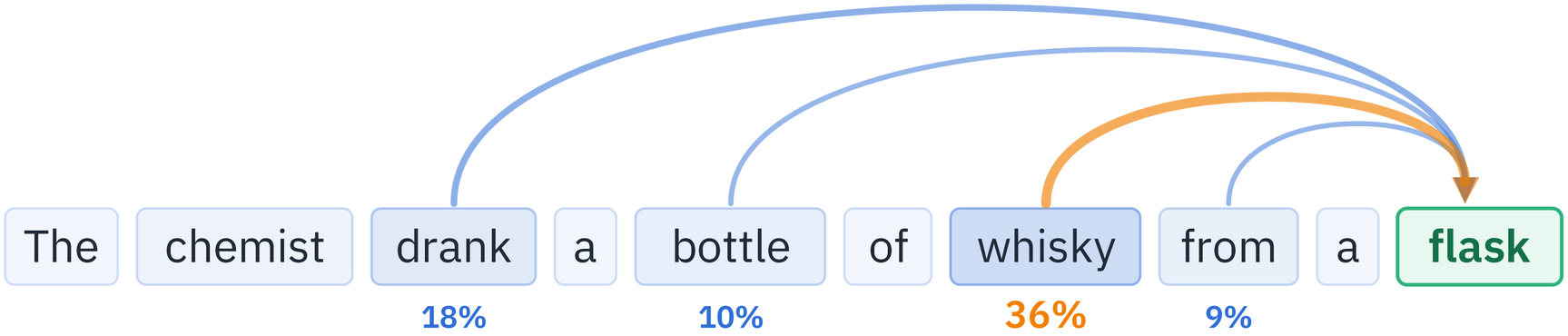
Our chemist, two sentences · GPT-2 small, layer 12, head 9



“flask” looks back, mostly at **heated**: lab glass

How alike are the two “flask” vectors?





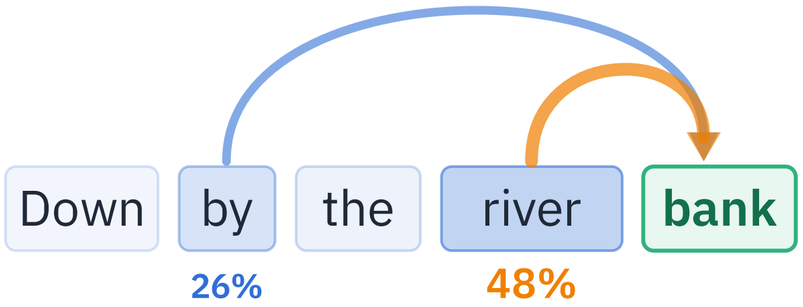
same head, other sentence: mostly **whisky**: a hip flask

The neighbours pull “flask” apart.

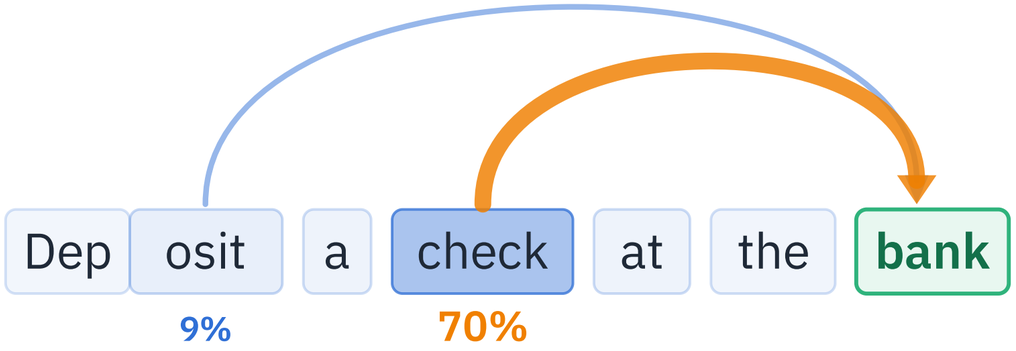
GPT-2 small (Radford et al., OpenAI, 2019), run by us

Attention in action: one word, two meanings

GPT-2 small, real numbers: layer 12, head 9

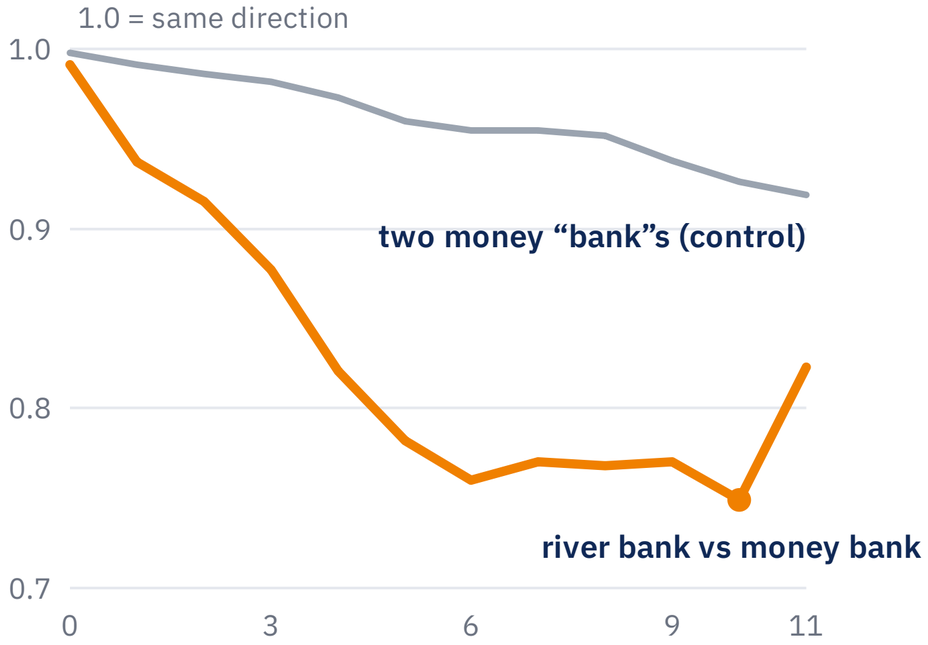


“bank” looks back and takes mostly **river**



same head, other sentence: mostly **check**

How alike are the two “bank” vectors?



Same word, same starting vector. Attention mixes in the neighbours, and “bank” becomes two different vectors.

GPT-2 small (Radford et al., OpenAI, 2019), run by us · the “bank” example is a classic of the attention literature

Two kinds of memory

Parameters

A vague recollection of something read months ago. Frozen at the training cutoff.

Context window

Working memory: whatever is in this conversation. Exact, and used directly.

Paste the document in, and it answers from the document.
Ask from memory, and it answers from the recollection.

2 SO IT FAILS BY stale, confident claims

A library API renamed after the cutoff still comes back, in confident code

Framing from Karpathy, Deep Dive into LLMs (2025)

LLM dreams

BASE MODEL

Run a base model freely: documents that never existed

Python code

```
import pandas as pd
from pandas.cleaning import smart_fill

def load_sensor_log(path):
    """Read a CSV log and fill gaps."""
    df = pd.read_csv(path)
    df = smart_fill(df, "physics")
    return df

# Tested with pandas 3.4.1
```

Amazon product page

Kitchen Science: 101 Experiments You Can Eat, by Dr. Hannah Moss

★★★★☆ 4.6 (12,847 ratings)
#3 Best Seller in Children's Science Books

Paperback $18.99 List: $24.99
312 pages, Brightline Press, 2019

ISBN-13: 978-1-4521-8836-2

Wikipedia article

Labrador Dog
The Labrador Dog is a British breed of retriever gun dog, developed from water dogs brought from Newfoundland in the early 1700s. Adults weigh 27–41 kg and live 11–14 years.

The breed was registered by the Kennel Club in 1897. It was the most popular dog breed in the United States from 1991 to 2022.

References
1. Hartley, J. (2004), pp. 212–219

Every document has the right form. The red parts are invented: plausible, not looked up.

A base model is not an assistant

BASE MODEL

It continues documents

What is the capital of France?

The base model replies:

What is the capital of Germany?
What is the capital of Italy?
What is the capital of Spain?
...

Your question looks like the start of a quiz page, so it writes more quiz page.
That is the correct behaviour for a document simulator.

Fine-tuning stage: Training an Assistant

BASE MODEL → ASSISTANT MODEL

Train on conversations instead of web pages

Both turns written by a person

```
<USER>
Can you explain the term "monopsony" in
economics?

<ASSISTANT>
"Monopsony" refers to a market with a
single buyer …
```

a human labeler, following long written instructions

Same training, different data.
The result is an **assistant model**: now the most likely continuation of a question is **an answer**.

About **100,000** conversations like this, written by people following long labeling instructions.

Today, increasingly, also by models.

You are talking to a simulation of a careful labeler.

Karpathy, Deep Dive into LLMs (2025)

Easier to judge than to write

Human preference: RLHF

Writing a good haiku is hard.
Picking the better of two is easy.

People rank answers. A second model learns to predict the ranking. The LLM is tuned to score well with that judge.

A

Steel curves intertwine,
holding life's pages in line,
quiet, small design.

B ✓

Silver arcs entwined,
holding life's pages aligned,
in steel we find bind.

3 SO IT FAILS BY agreeableness

Rewarded for answers people liked, so it tends to agree with you, even when you are wrong

Karpathy (2023) · Sharma et al. (2023) · OpenAI GPT-4o sycophancy rollback (Apr 2025)

How to train your assistant, 2026

2023 had two stages. Today there are three.

1 · Pretraining

The internet → a base model.

Expensive: months on thousands of GPUs. Done rarely.

`like reading every textbook`

2 · Fine-tuning

Conversations → an assistant.

Cheap: about a day. Can be re-run weekly.

`like studying worked examples`

3 · Reinforcement

RLHF **Reinforcement Learning from Human Feedback**

Practice problems with checkable answers, plus human preference.

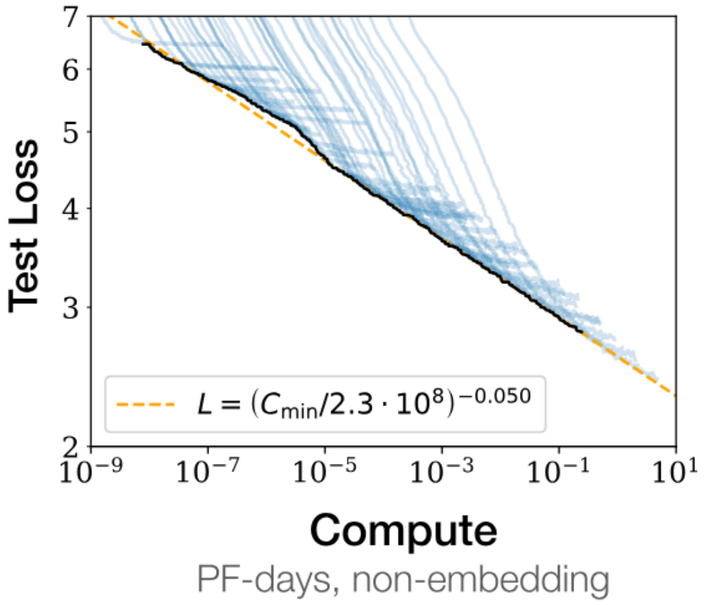
Stage 3 is new since 2023, and it is where reasoning comes from.

Karpathy, Deep Dive into LLMs (2025): the textbook analogy

Scaling laws

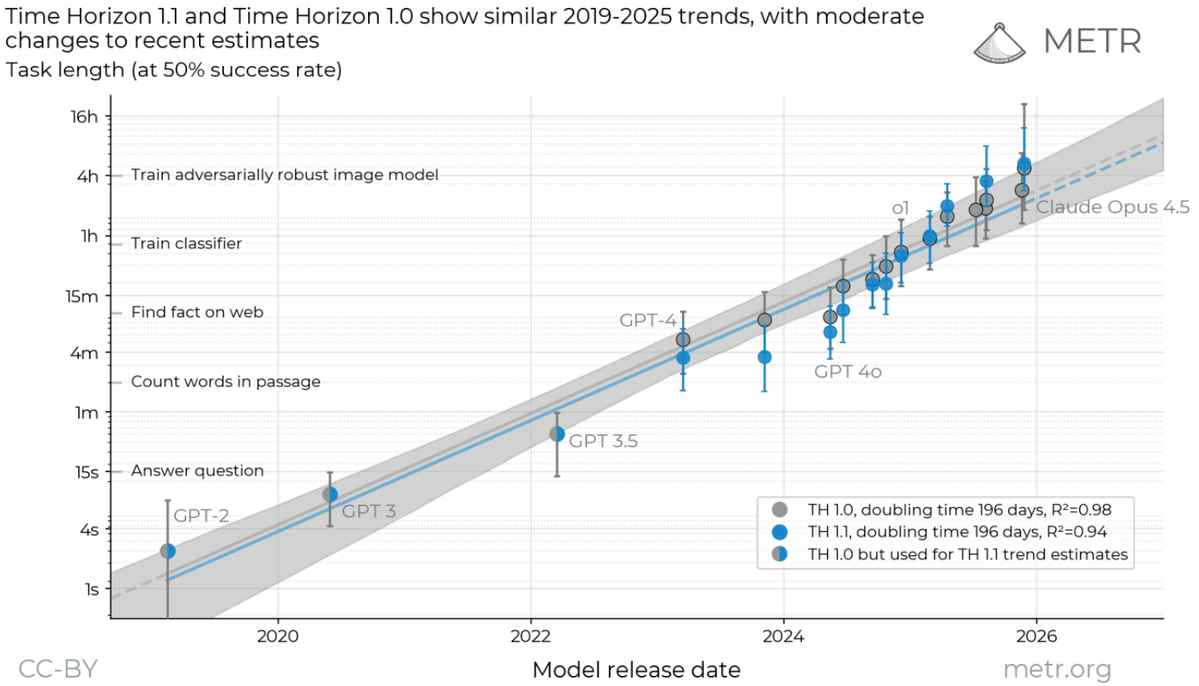
More compute, predictably better, and still on trend

The law (Kaplan et al., 2020)



Spend 10× more compute and the error drops by about 11%, predictably, every time.

What it buys (METR, 2026)



The length of task an AI agent can finish doubles every ~7 months; since 2024, every ~4.

Kaplan et al., Scaling Laws for Neural Language Models (2020), Fig. 1 · METR, Time Horizon 1.1 (Jan 2026), CC-BY

Jagged capability

Superhuman next to subhuman, in the same model

Can	Has failed at
Gold-medal level at the International Mathematical Olympiad (2025)	"Which is bigger, 9.11 or 9.9?" Counting the dots in a picture

Skills follow the training data and the checkable rewards, not human difficulty.
Swiss cheese: solid, with holes you cannot see from outside.

4 SO IT FAILS BY uneven capability

Success on a hard task tells you nothing about the easy one next to it. Test on your task.

OpenAI and Google DeepMind, IMO 2025 · "Swiss cheese": Karpathy, Deep Dive (2025)

The context window

A finite buffer, not a memory

2023 · GPT-4

8,000 tokens

about 15 pages

2026

200,000 to 1 M+ tokens

a few books. Bigger, but not perfect.

As it fills, the middle gets less attention and early instructions fade. No error. No warning.

5

SO IT FAILS BY long-session degradation

Two hours in, it "forgets" a constraint you set at the start

Liu et al., Lost in the Middle (2023) · Chroma, Context Rot (2025)

Tools turn a model into an agent

2023: a demo

A chatbot searched the web, built a table, used a calculator, plotted with Python, extrapolated.

2026: the default

The model writes a tool call. The harness runs it and puts the result back in the window. Repeat.



An agent

search · run code · read files · call a simulator · see images

Every tool result is **working memory**, not recollection. That is why tools help.

REFERENCE

Five mechanics, five failures

	MECHANIC	FAILURE	SO YOU
1	**Next-token prediction**	**Fluent wrongness**	recompute every number
2	**Frozen parameters**	**Stale, confident claims**	put current docs in the window
3	**Human-preference tuning**	**Agreeableness**	don't argue it into your answer
4	**Scaling and RL on checkable tasks**	**Uneven capability**	test on your task
5	**Finite context window**	**Long-session degradation**	manage the window (this week)

Structural, therefore architectural

These failures are built in.
A better prompt does not remove them.

So keep the model **out of the numbers**:
it proposes, the tested ground computes, it reads the result.

That is Approach C, from last week. Now you know **why**.

Outline

01	Field update	what moved this week
02	LLM	what it is, how it is trained, how it fails
03	**More git**	worktrees, merge conflicts, GitHub ←

git: the photo album

a commit is a photograph of your project

your folder

README.md notes.txt

the album = history




```
$ git switch --detach b3ced92
HEAD is now at b3ced92 Add README
```

Back to the first photo: sim.py did not exist yet.

You can put your folder back exactly as in any photo.

frame = staging · **photo** = commit · **album** = history

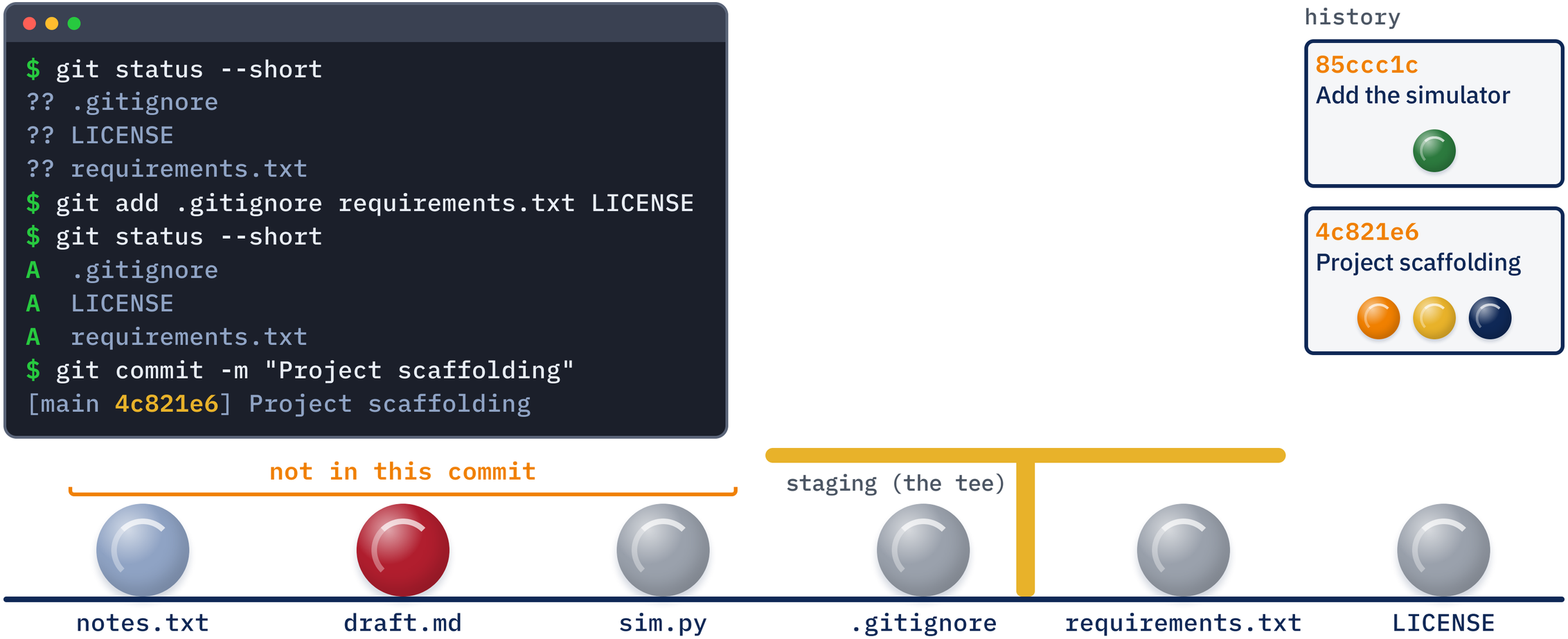
git: staging, the golf tee

staging is choosing what goes into the next commit

```
$ git status --short
?? .gitignore
?? LICENSE
?? requirements.txt
$ git add .gitignore requirements.txt LICENSE
$ git status --short
A  .gitignore
A  LICENSE
A  requirements.txt
$ git commit -m "Project scaffolding"
[main 4c821e6] Project scaffolding
```

history

85ccc1c
Add the simulator



4c821e6
Project scaffolding

One swing, one commit.

Unlike golf, the files stay on the floor: a commit is a copy.

git, for real: commits on main

your folder · the terminal · the history

your folder **main**

reactor-study/
- README.md
- sim.py

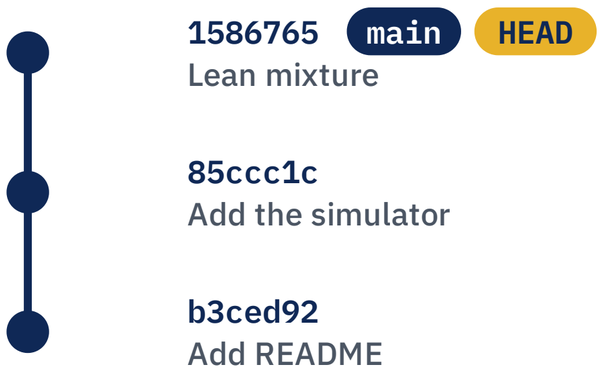
sim.py

```
T_IN = 1200
PHI = 0.9

def simulate(t_in, phi):
    ...
```

```
$ git init
$ git add README.md
$ git commit -m "Add README"
[main (root-commit) b3ced92] Add README
1 file changed, 1 insertion(+)
create mode 100644 README.md
$ git add sim.py
$ git commit -m "Add the simulator"
[main 85ccc1c] Add the simulator
1 file changed, 5 insertions(+)
create mode 100644 sim.py
$ git add sim.py
$ git commit -m "Lean mixture"
[main 1586765] Lean mixture
1 file changed, 1 insertion(+), 1
deletion(-)
```

history (git log)



git add chooses. git commit records. HEAD follows the newest commit.

git, for real: a branch

same sequence, one branch further

your folder **feature**

```
reactor-study/
  README.md
  sim.py
  plot.py
```

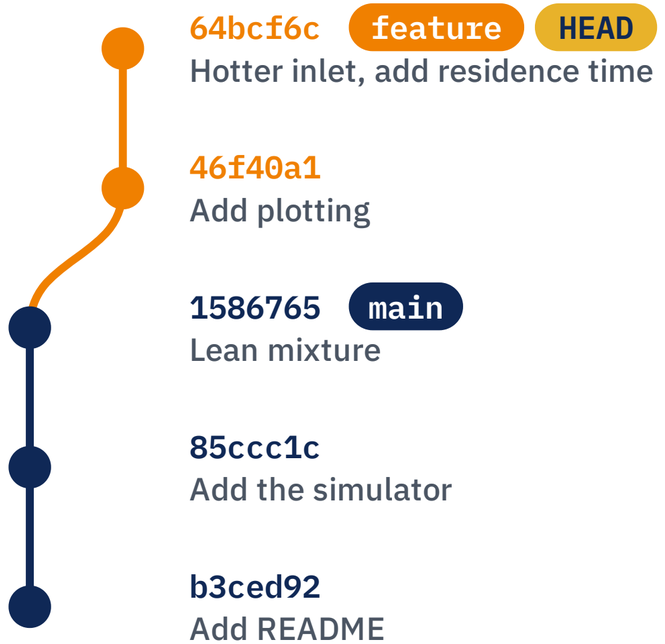
sim.py

```
T_IN = 1300
PHI = 0.9
TAU = 0.05

def simulate(t_in, phi):
    ...
```

```
[main 1586765] Lean mixture
$ git switch -c feature
Switched to a new branch 'feature'
$ git add plot.py
$ git commit -m "Add plotting"
[feature 46f40a1] Add plotting
1 file changed, 4 insertions(+)
create mode 100644 plot.py
$ git add sim.py
$ git commit -m "Hotter inlet, add
residence time"
[feature 64bcf6c] Hotter inlet, add
residence time
1 file changed, 2 insertions(+), 1
deletion(-)
```

history (git log)



A branch is a name. main stayed at 1586765 while feature moved on.

git: the switch

the folder changes under your feet

your folder **feature**

```
reactor-study/
  README.md
  sim.py
  plot.py
```

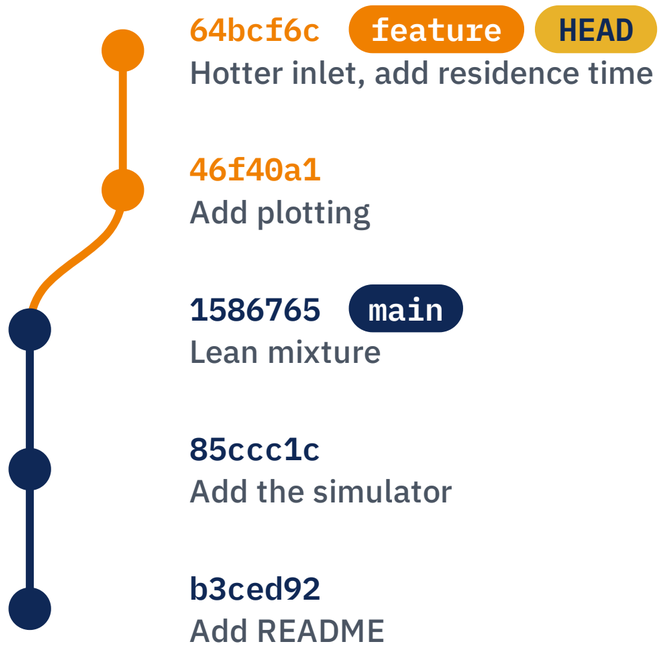
sim.py

```
T_IN = 1300
PHI = 0.9
TAU = 0.05

def simulate(t_in, phi):
    ...
```

```
$ git switch main
Switched to branch 'main'
$ ls
README.md
sim.py
$ git switch feature
Switched to branch 'feature'
$ ls
README.md
plot.py
sim.py
```

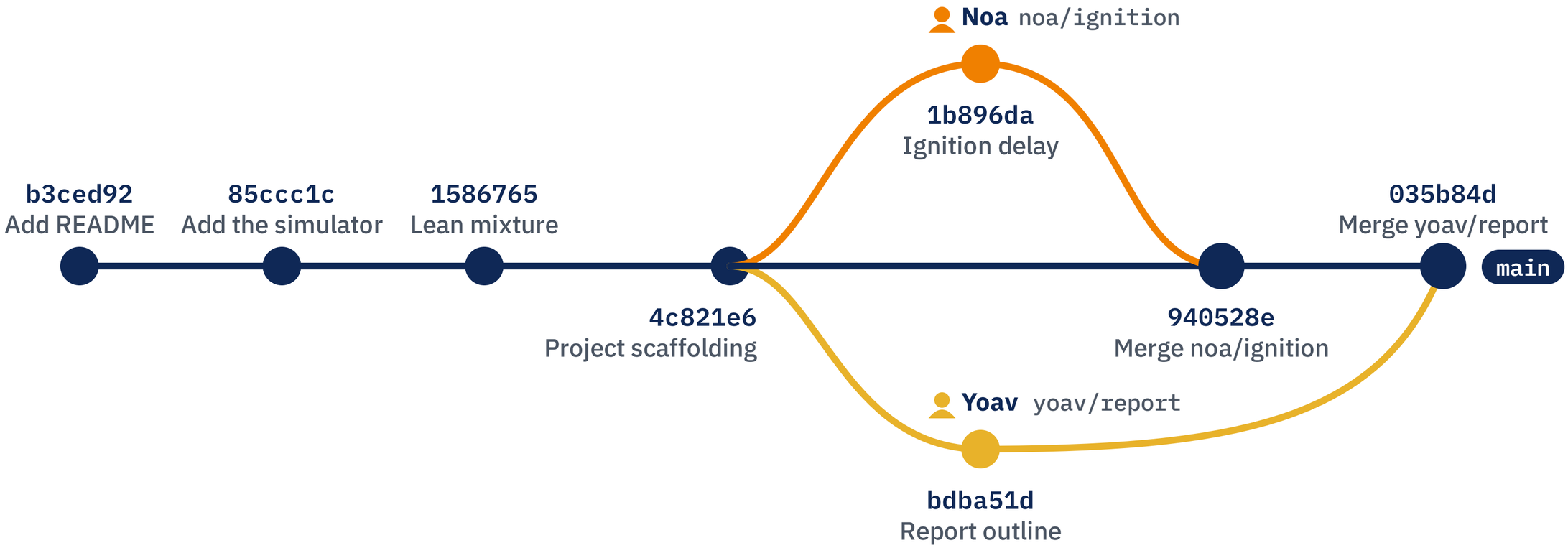
history (git log)



Same folder. Different branch, different files.

git: working as a team

two people, two branches, one main



Everyone works on a branch. main only moves by merging.

On GitHub, a merge into main is a **pull request**. Merge **conflicts**: Week 2.

git: two jobs, one folder

an urgent fix arrives while your work is half-done

your folder **feature**

- reactor-study/
  - README.md
  - sim.py
  - plot.py

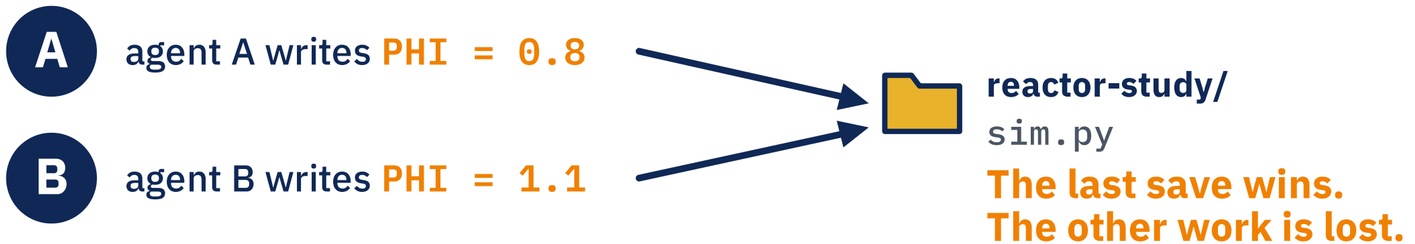
sim.py

```
T_IN = 1300
PHI = 0.95
TAU = 0.05
```

half-done edit
not committed yet

```
$ git status -s
 M sim.py
$ git switch main
error: Your local changes to the following files would be overwritten by checkout:
        sim.py
Please commit your changes or stash them before you switch branches.
Aborting
```

two AI agents, one folder



One folder shows one branch at a time, so two jobs collide.

git worktree: a second folder

one history, two places to work

```
$ git worktree add ../reactor-study-fix -b fix main
Preparing worktree (new branch 'fix')
HEAD is now at 1586765 Lean mixture
$ git worktree list
/home/you/reactor-study      64bcf6c [feature]
/home/you/reactor-study-fix  1586765 [fix]
```

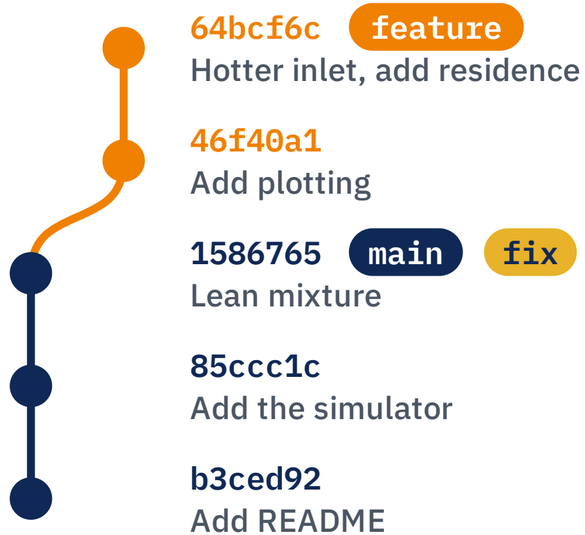
folder 1 — feature

- reactor-study/
  - README.md
  - sim.py
  - plot.py

folder 2 — fix

- reactor-study-fix/
  - README.md
  - sim.py

one shared history (git log)



Two folders, one history. Each folder shows its own branch.

Like a second desk for the same project. Nothing is copied: the album stays one.

Work in both, then clean up

a commit in one folder shows up in the one history

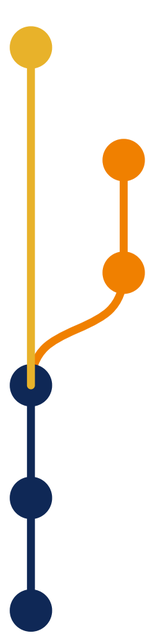
```
$ cd ../reactor-study-fix
$ git commit -am "Fix README typo"
[fix c0a7f31] Fix README typo
 1 file changed, 1 insertion(+), 1 deletion(-)
$ cd ../reactor-study
$ git worktree remove ../reactor-study-fix
```

folder 1 — feature

- reactor-study/
  - README.md
  - sim.py
  - plot.py

folder removed
the fix stays in history

one shared history (git log)



c0a7f31 **fix**
Fix README typo

64bcf6c **feature**
Hotter inlet, add residence time

46f40a1
Add plotting

1586765 **main**
Lean mixture

85ccc1c
Add the simulator

b3ced92
Add README

The fix lands in the same history. Then the extra folder can go.

Each AI agent gets its own worktree, so none overwrites another. This is how the course runs agents.

Merging: usually automatic

two branches changed different files

```
$ git merge noa/ignition
Merge made by the 'ort' strategy.
 sim.py | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)
```

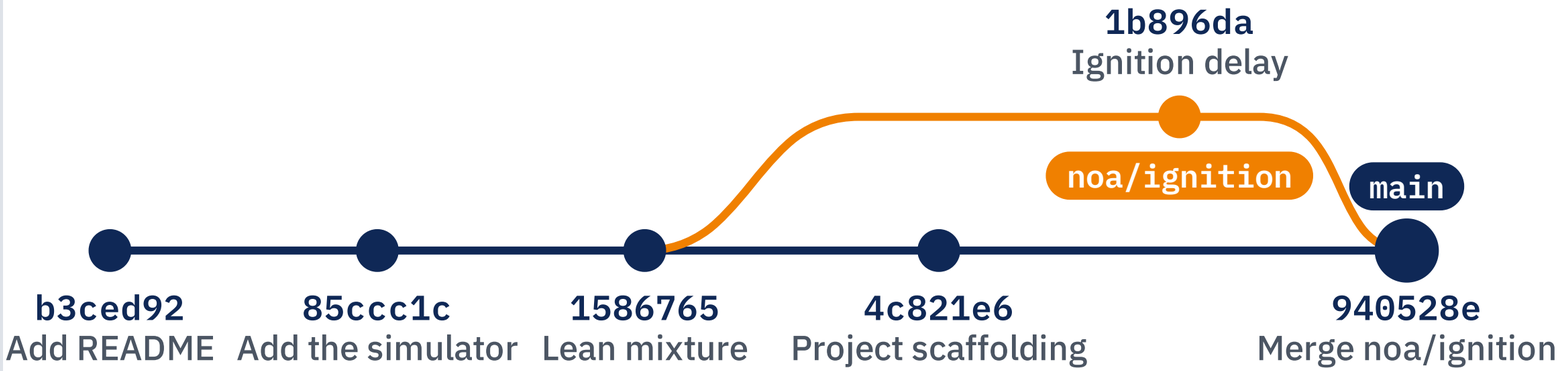
who changed what

README.md — main

sim.py — noa/ignition

Different files. **Nothing to decide.**

history (git log)



Different files: git merges them on its own.

Most merges look like this. No questions, one new merge commit that joins the two lines.

A merge conflict

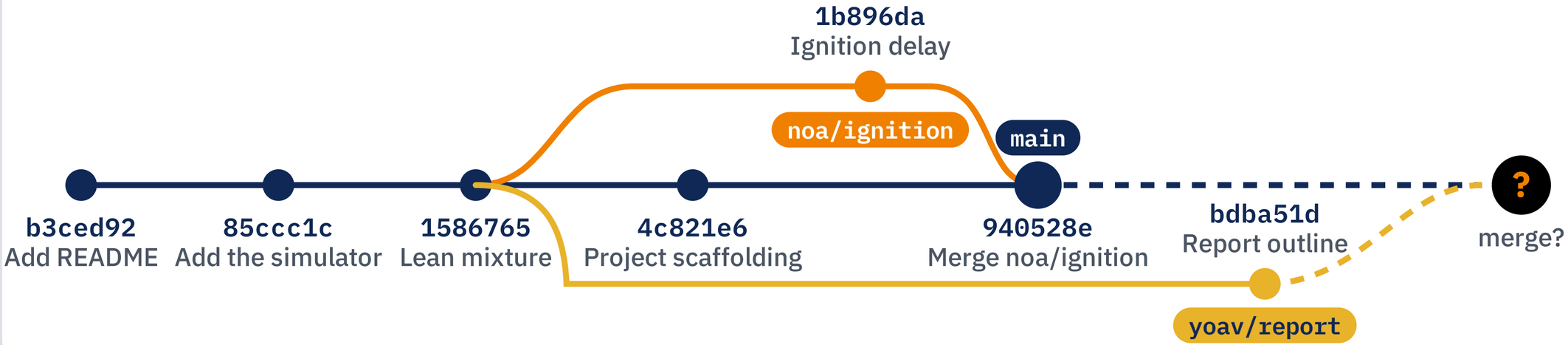
both branches changed the same line

```
$ git merge yoav/report
Auto-merging sim.py
CONFLICT (content): Merge conflict in sim.py
Automatic merge failed; fix conflicts and then commit the result.
```

sim.py

```
<<<<<<< HEAD
PHI = 1.1          ← Noa (main)
=======
PHI = 0.8          ← Yoav
>>>>>>> yoav/report
```

history (git log)



Same line, two answers. git cannot choose for you.

The markers show both versions. Nothing is lost yet: it is a question, waiting for you.

Resolving it

a conflict is a question, not an error

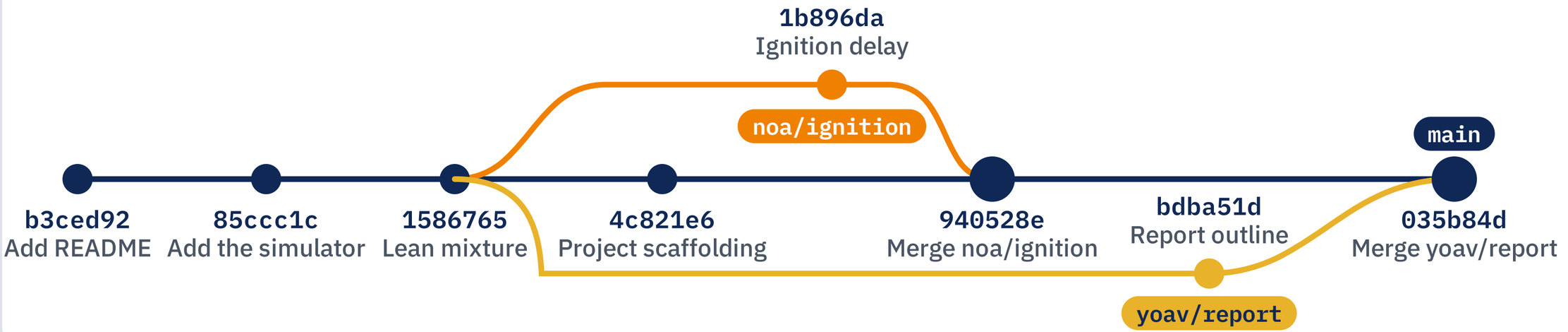
```
$ git status -s
UU sim.py
$ git add sim.py
$ git commit
[main 035b84d] Merge branch 'yoav/report'
```

sim.py

```
PHI = 0.95        ← you decide
```

Pick one value, or **combine them**.
No markers left.

history (git log)

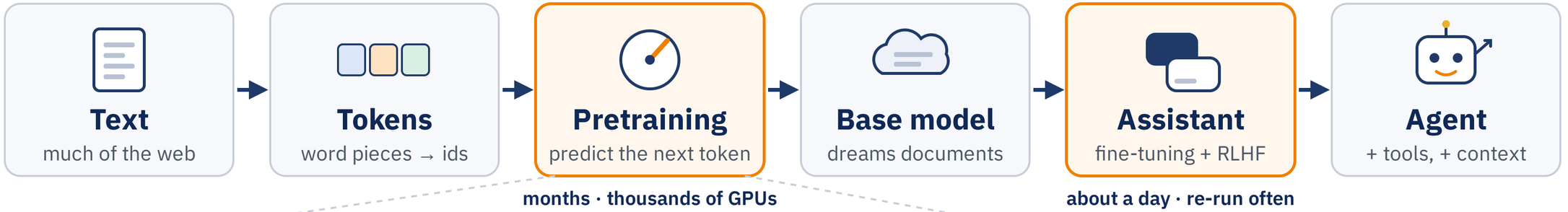


You answer once. The merge commit joins both lines.

Edit the file, delete the markers, then git add and git commit. A conflict is a question, not an error.

Today, in one picture

LLM: HOW IT IS MADE



"bank": by the river,
or with a check?
Attention decides.

...AND HOW IT FAILS

1. **fluent wrongness** — from next-token prediction
2. **stale, confident claims** — from frozen parameters
3. **agreeableness** — from preference tuning
4. **uneven capability** — from scaling, RL
5. **long-session degradation** — from finite context window

GIT: WORKING SAFELY, IN PARALLEL



Use it for what it is good at.
Check what it is bad at.

NEXT LECTURE

Week 3: The harness

An agent is a harness plus a model: the loop, tools, memory. Skills and hooks. Model selection and cost. The 4 Ds.